

Looping

Programming for Problem Solving (PPS)

GTU # 3110003

USING

{C}

Programming



Prof. Nilesh Gambhava

Computer Engineering Department,
Darshan Institute of Engineering & Technology, Rajkot





Life is all about Repetition.

We do same thing everyday



What is loop?

- ▶ Loop is used to execute the block of code several times according to the condition given in the loop. It means it executes the same code multiple times.

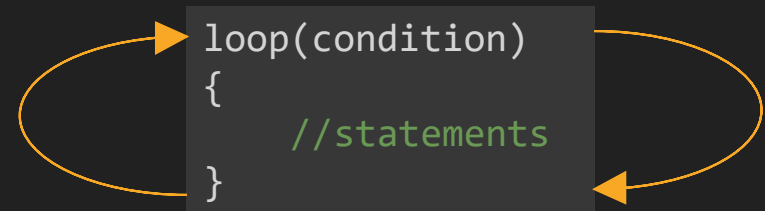
"Hello"

5

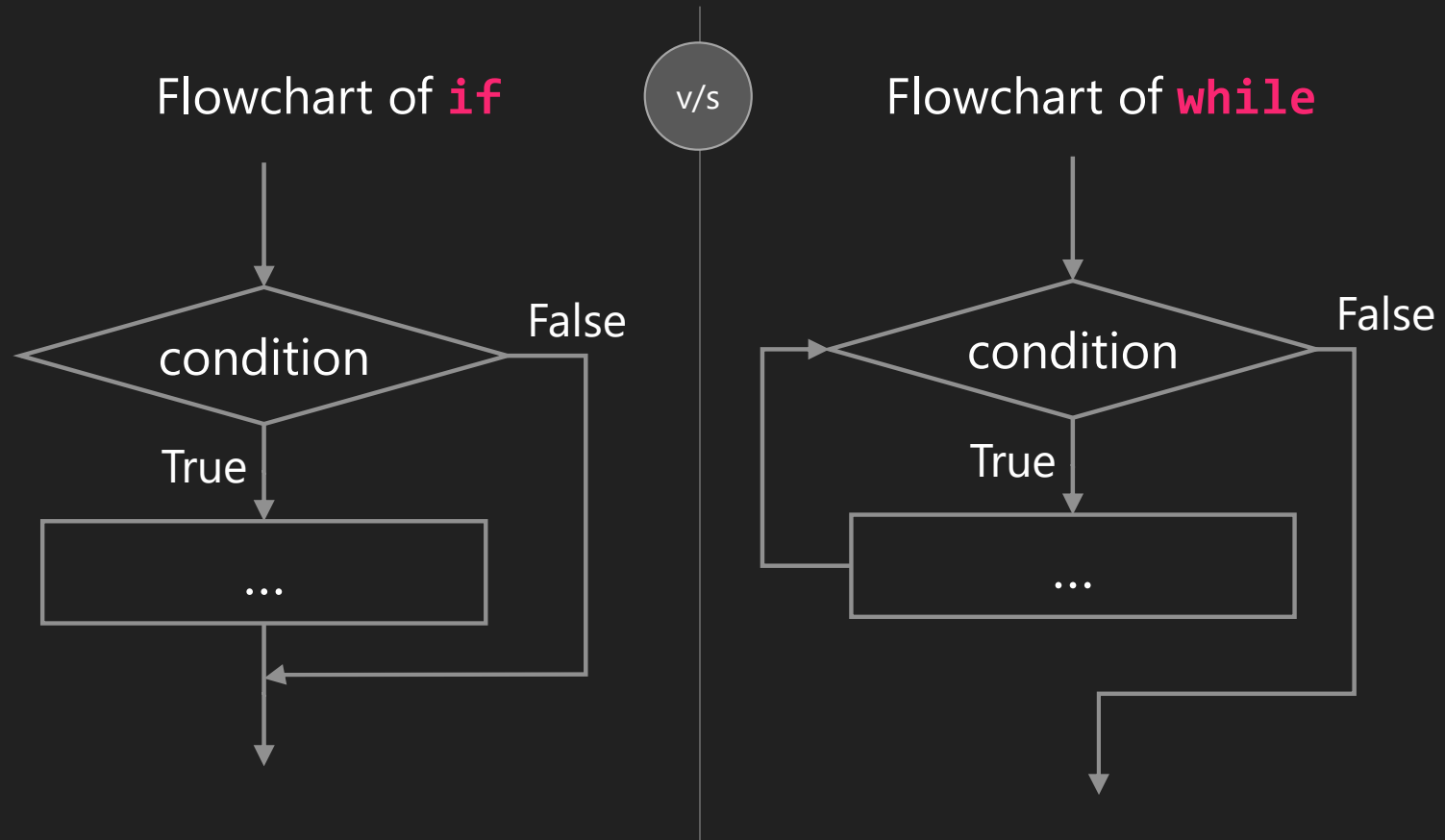
```
printf("Hello\n");  
printf("Hello\n");  
printf("Hello\n");  
printf("Hello\n");  
printf("Hello\n");
```

Output

```
Hello  
Hello  
Hello  
Hello  
Hello
```



if v/s while



Looping or Iterative Statements in C

Looping Statements are

Entry Controlled Loop:	<code>while, for</code>
Exit Controlled Loop:	<code>do...while</code>
Virtual Loop:	<code>goto</code>



While loop



While Loop

- ▶ **while** is an entry controlled loop
- ▶ Statements inside the body of **while** are repeatedly executed till the condition is true
- ▶ **while** is keyword

Syntax

```
while(condition)
{
    // Body of the while
    // true part
}
```

WAP to print 1 to n(while loop)

Program

```
1 #include <stdio.h>
2 void main()
3 {
4     int i,n;
5     i=1;
6     printf("Enter n:");
7     scanf("%d",&n);
8     while(i<=n)
9     {
10         printf("%d\n",i);
11         i=i+1;
12     }
13 }
```

Output

```
Enter n:10
1
2
3
4
5
6
7
8
9
10
```

WAP to print Odd numbers between 1 to n(while loop)

Program

```

1  #include<stdio.h>
2  void main()
3  {
4      int i=1, n;
5      printf("Enter the value of n");
6      scanf("%d",&n);
7      while(i <= n)
8      {
9          if(i%2!=0)
10             printf("%d/n",i);
11             i=i+1;
12     }
13 }

```

Output

```

Enter the value of n
5
1
3
5

```

Line	i	n	condition	Process	Output
1				Linking header file	
2				Calling void main()	
3					
4	1			Declarations	
5					Enter value of n
6		5		Scanning value of n	5
7			i <= n, TRUE	While loop condition	
8					
9			i%2 != 0 TRUE	If condition	
10				Printing value of i	1
11	2			Incrementing value of i by 1	
12				Jump to line:7	
7			i <= n, TRUE	While loop condition	
8					
9			i%2 != 0 FALSE	If condition, skip line:10	
11	3			Incrementing value of i by 1	
12				Jump to line:7	
7			i <= n, TRUE	While loop condition	
8					
9			i%2 != 0 TRUE	If condition	
10				Printing value of i	3
11	4			Incrementing value of i by 1	
12				Jump to line:7	

13				End of program	
----	--	--	--	----------------	--

WAP to print multiplication table(while loop)

Program

```
1  #include<stdio.h>
2  void main()
3  {
4      int i=1,n;
5      printf("Enter n for multiplication table:");
6      scanf("%d",&n);
7      while(i<=10)
8      {
9          printf("%d * %d = %d\n",n,i,n*i);
10         i=i+1;
11     }
12 }
```

Output

```
Enter n for multiplication table:5
5 * 1 = 5
5 * 2 = 10
5 * 3 = 15
5 * 4 = 20
5 * 5 = 25
5 * 6 = 30
5 * 7 = 35
5 * 8 = 40
5 * 9 = 45
5 * 10 = 50
```

WAP to Sum of 5 numbers entered by user(while loop)

Program

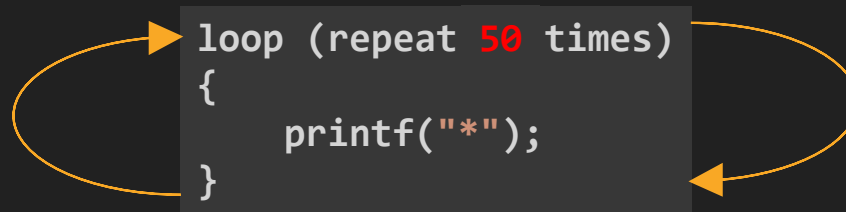
```
1  #include<stdio.h>
2  void main()
3  {
4      int sum=0, i=1,n;
5      while(i<=5)
6      {
7          printf("Enter a number=");
8          scanf("%d",&n);
9          sum=sum+n;
10         i=i+1;
11     }
12     printf("Sum is=%d",sum);
13 }
```

Output

```
Enter a number=10
Enter a number=20
Enter a number=30
Enter a number=40
Enter a number=50
Sum is=150
```

Looping

- ▶ Loop Control in C, is the repeated execution of code, for a **specified number of times**, if the **condition is met**.



Syntax and Logic

Swimming Rules

1. Breath control
2. Kicking legs
3. Back stroke with arms
4. Front stroke with arms
5. Crawling in water

To Swim



Syntax

```
while(condition)
{
    // Body of the while
    // true part
}
```

Logic

```
int i = 1;
while (i <= 5)
{
    printf("%d\n", i);
    i=i+1;
}
```

How to build logic? Step-1

Step 1: Understand the problem statement

- ▶ e.g. Write a program to find factors of a number.
- ▶ Run following questions through mind
- ▶ What is the factor of a number?
 - Factor is a number that divides another number evenly with no remainder.
 - For example, 1,2,3,4,6,12 are factors of 12.
- ▶ How many variables needed? What should be their data types?(Inputs/Outputs)
 - To get number from user we need variable **n**.
 - Now we need to divide **n** with 1,2,3,...,n. For this we will declare a loop variable **i** initialized as 1.
 - Both variables should be of **integer** data type.
- ▶ What control structure you require?
 - First we need a **loop** to divide **n** by 1,2,3,...,n, loop will start from 1 and ends at **n**.
 - Inside loop we need **if structure** to check **n%i==0** (Number n is evenly divisible by **i** or not).

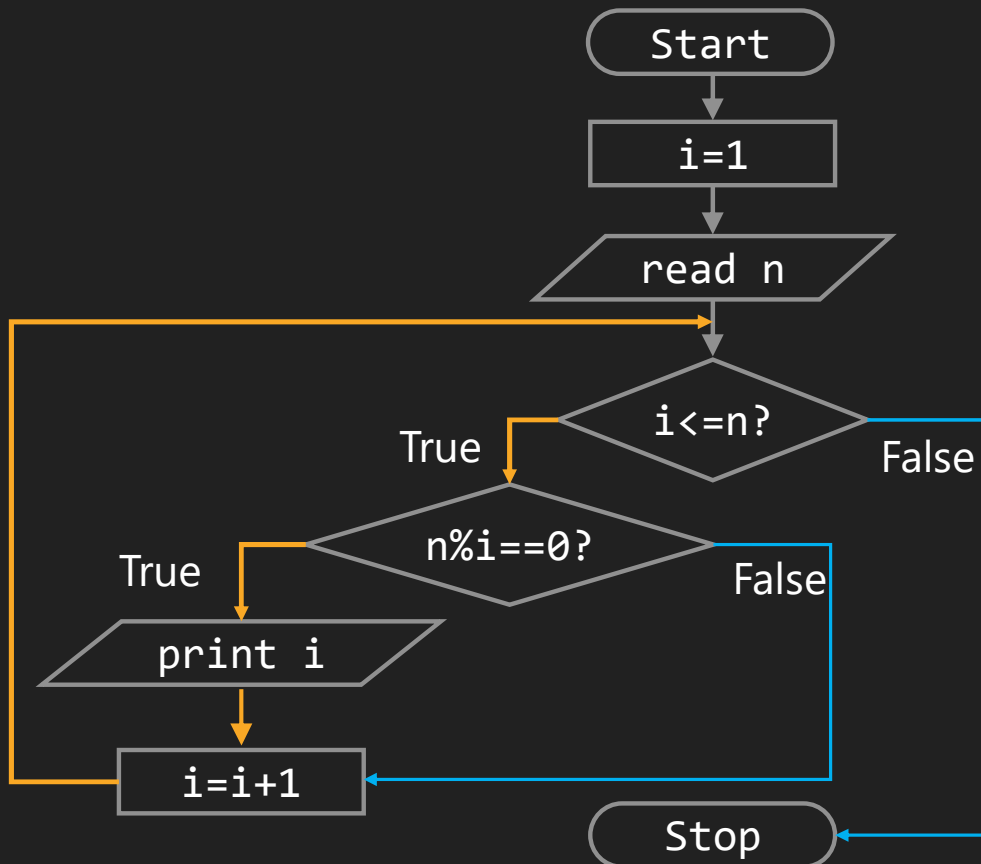
How to build logic? Step-2

Step 2: Think for 1 or 2 examples

- ▶ Consider $n=6$, now take $i=1$
 - ➔ $6\%1==0$, TRUE; So, 1 is factor of 6
 - ➔ $6\%2==0$, TRUE; So, 2 is factor of 6
 - ➔ $6\%3==0$, TRUE; So, 3 is factor of 6
 - ➔ $6\%4==2$, FALSE; So, 4 is not factor of 6
 - ➔ $6\%5==1$, FALSE; So, 5 is not factor of 6
 - ➔ $6\%6==0$, TRUE; So, 6 is factor of 6
- ▶ From this we can infer that loop variable i starts with 1 and incremented by one for next iteration then ends at value n .
- ▶ Consider $n=10$, factors are 1, 2, 5, 10
- ▶ Consider $n=11$, factor is 1, 11
- ▶ From this we can infer that 1 and number itself are always factors of any number n .

How to build logic? Step-3

Step 3: Draw flowchart/steps on paper or in mind



Steps

Step 1: Start
Step 2: Declare variables n, i
Step 3: Initialize variable
 $i \leftarrow 1$
Step 4: Read value of n
Step 5: Repeat the steps until $i = n$
 5.1: if $n \% i == 0$
 Display i
 5.2: $i = i + 1$
Step 7: Stop

How to build logic? Step-4

Step 4: Writing Pseudo-code

- ▶ Pseudo-code is an informal way to express the design of a computer program or an algorithm.
- ▶ It does not require any strict programming language syntax.

Pseudo-code

```
Initialize i=1 integer
Declare n as integer
Input n
while i<n
    if n%i
        print i
    end if
    increment i=i+1
end while
```

WAP to find factors of a number(while loop)

Program

```
1  #include <stdio.h>
2  void main()
3  {
4      int i=1,n;
5      printf("Enter n to find factors=");
6      scanf("%d",&n);
7      while(i<=n)
8      {
9          if(n%i==0)
10             printf("%d,",i);
11             i=i+1;
12     }
13 }
```

Output

```
Enter n to find factors=12
1,2,3,4,6,12,
```

WAP to print reverse a number(while loop)

Program

```
1  #include <stdio.h>
2  void main()
3  {
4      int n;
5      printf("Enter a number=");
6      scanf("%d",&n);
7      while(n!=0)
8      {
9          printf("%d",n%10);
10         n=n/10;
11     }
12 }
```

Output

```
Enter a number=1234
4321
```

WAP to check given number is perfect or not(while loop)

```
1 void main(){
2     int i=1,n,sum=0;
3     printf("Enter a number:");
4     scanf("%d",&n);
5     while(i<n)
6     {
7         if(n%i==0)
8         {
9             printf("%d+",i);
10            sum=sum+i;
11        }
12        i=i+1;
13    }
14    printf("=%d",sum);
15    if(sum==n)
16        printf("\n%d is a perfect number",n);
17    else
18        printf("\n%d is not a perfect number",n);
19 }
```

Output

Enter a number:6
1+2+3=6
6 is a perfect number

Output

Enter a number:8
1+2+4+=7
8 is not a perfect number

Output

Enter a number:496
1+2+4+8+16+31+62+124+248+=496
496 is a perfect number

WAP to check given number is prime or not(while loop)

```
1 void main()
2 {
3     int n, i=2, flag=0;
4     printf("Enter a number:");
5     scanf("%d",&n);
6     while(i<=n/2)
7     {
8         if(n%i==0)
9         {
10             flag=1;
11             break;
12         }
13         i++;
14     }
15     if (flag==0)
16         printf("%d is a prime number",n);
17     else
18         printf("%d is not a prime number",n);
19 }
```

Output

Enter a number:7
7 is a prime number

Output

Enter a number:9
9 is not a prime number



for loop



for Loop

- ▶ **for** is an entry controlled loop
- ▶ Statements inside the body of **for** are repeatedly executed till the condition is true
- ▶ **for** is keyword

Syntax

```
for (initialization; condition; updateStatement)
{
    // statements
}
```

- ▶ The initialization statement is executed **only once**.
- ▶ Then, the condition is evaluated. If the condition is **false**, the **for** loop is **terminated**.
- ▶ If the condition is **true**, statements inside the body of for loop are executed, and the update statement is updated.
- ▶ Again the condition is evaluated.

WAP to print numbers 1 to n (for loop)

Program

```
1 #include<stdio.h>
2 void main()
3 {
4     int i,n;
5     printf("Enter a number:");
6     scanf("%d",&n);
7     for(i=1;i<=n;i++)
8     {
9         printf("%d\n",i);
10    }
11 }
```

Output

```
Enter a number:5
1
2
3
4
5
```

WAP to find factors of a number (for loop)

Program

```
1  #include <stdio.h>
2  void main()
3  {
4      int i,n;
5      printf("Enter n to find factors=");
6      scanf("%d",&n);
7      for(i=1;i<=n;i++)
8      {
9          if(n%i==0)
10             printf("%d,",i);
11     }
12 }
```

Output

```
Enter n to find factors=12
1,2,3,4,6,12,
```

WAP to check given number is perfect or not(for loop)

```
1 void main(){
2     int i,n,sum=0;
3     printf("Enter a number:");
4     scanf("%d",&n);
5     for(i=1;i<n;i++)
6     {
7         if(n%i==0)
8         {
9             printf("%d+",i);
10            sum=sum+i;
11        }
12    }
13    printf("=%d",sum);
14    if(sum==n)
15        printf("\n%d is a perfect number",n);
16    else
17        printf("\n%d is not a perfect number",n);
18 }
```

Output

Enter a number:6
1+2+3=6
6 is a perfect number

Output

Enter a number:8
1+2+4+=7
8 is not a perfect number

Output

Enter a number:496
1+2+4+8+16+31+62+124+248+=496
496 is a perfect number



do while loop



do while Loop

- ▶ `do while` is an exit controlled loop.
- ▶ Statements inside the body of `do while` are repeatedly executed till the condition is true.
- ▶ `Do` and `while` are keywords.

Syntax

```
do
{
    // statement
}
while (condition);
```

- ▶ Loop body will be executed **first**, and then condition is checked.
- ▶ If the condition is **true**, the body of the loop is executed again and the condition is evaluated.
- ▶ This process goes on until the condition becomes **false**.
- ▶ If the condition is false, the loop ends.

WAP to print Odd numbers between 1 to n(do while loop)

Program

```
1 void main()
2 {
3     int i=1,n;
4     printf("Enter a number:");
5     scanf("%d",&n);
6     do
7     {
8         if(i%2!=0)
9         {
10             printf("%d,",i);
11         }
12         i=i+1;
13     }
14     while(i<=n);
15 }
```

Output

```
Enter a number:5
1,3,5
```

WAP to find factors of a number(do while loop)

Program

```
1 void main()
2 {
3     int i=1,n;
4     printf("Enter a number:");
5     scanf("%d",&n);
6     do
7     {
8         if(n%i==0)
9         {
10             printf("%d,",i);
11         }
12         i=i+1;
13     }
14     while(i<=n);
15 }
```

Output

```
Enter a number:6
1,2,3,6,
```

WAP to print reverse a number(do while loop)

Program

```
1 void main()
2 {
3     int n;
4     printf("Enter a number:");
5     scanf("%d",&n);
6     do
7     {
8         printf("%d",n%10);
9         n=n/10;
10    }
11    while(n!=0);
12 }
```

Output

```
Enter a number=1234
4321
```



goto statement



goto Statement

- ▶ `goto` is an virtual loop
- ▶ The `goto` statement allows us to transfer control of the program to the specified `label`.
- ▶ `goto` is keyword

Syntax

```
goto label;  
.  
.  
.  
label:
```

Syntax

```
label:  
.  
.  
.  
goto label;
```

- ▶ The `label` is an identifier. When the `goto` statement is encountered, the control of the program jumps to `label:` and starts executing the code.

WAP to print Odd numbers between 1 to n(goto)

Program

```
1 void main()
2 {
3     int i=1,n;
4     printf("Enter a number:");
5     scanf("%d",&n);
6     odd:
7     if(i%2!=0)
8     {
9         printf("%d,",i);
10    }
11    i=i+1;
12    if(i<=n)
13    {
14        goto odd;
15    }
16 }
```

Output

```
Enter a number:5
1,3,5
```

WAP to find factors of a number(goto)

Program

```
1 void main()
2 {
3     int i=1,n;
4     printf("Enter a number:");
5     scanf("%d",&n);
6     odd:
7     if(n%i==0)
8     {
9         printf("%d,",i);
10    }
11    i=i+1;
12    if(i<=n)
13    {
14        goto odd;
15    }
16 }
```

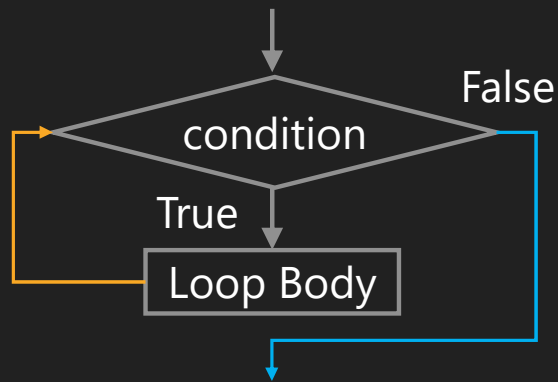
Output

```
Enter a number:6
1,2,3,6,
```

Types of loops

Entry Control Loop

```
int i=1;
while(i<=10)
{
    printf("%d",i++);
}
```

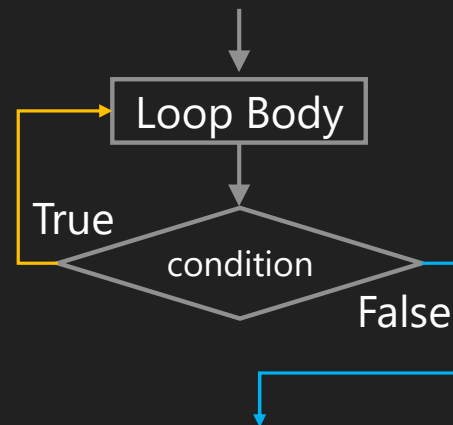


Entry Control Loop

```
int i;
for(i=1;i<=10;i++)
{
    printf("%d",i);
}
```

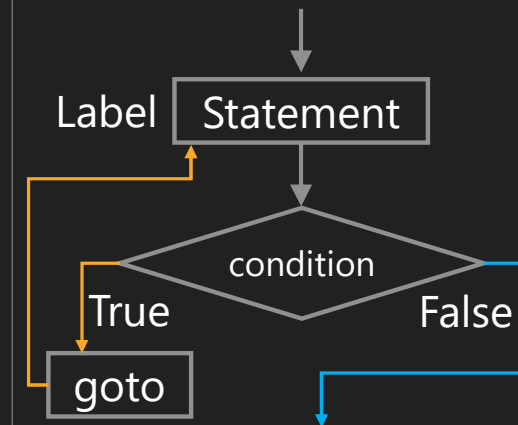
Exit Control Loop

```
int i=1;
do
{
    printf("%d",i++);
}
while(i<=10);
```



Virtual Loop

```
int i=1;
labelprint:
    printf("%d",i++);
    if(i<=10)
        goto labelprint;
```





Pattern

Always detect pattern in pattern



Pattern

There are important points to note in pattern

1. Determine, how many rows?
2. Determine, how many numbers/characters/columns in a row?
3. Determine, Increment/Decrement among the number of rows.
4. Determine, starting in each row

```
1
11
111
1111
11111
```

```
1
12
123
1234
12345
```

```
1
23
456
78910
```

```
  *
 * *
* * *
* * * *
 * * *
  * *
```

WAP to print given pattern (nested loop)

```
*  
**  
***  
****  
*****
```

No. of rows: 5

No. of characters

Row-1: *

Row-2: **

Row-3: ***

Row-4: ****

Row-5: *****

Inner loop: Increment
Outer loop: Increment

Starting: *

Program

```
1 void main()  
2 {  
3     int i,j;  
4     for(i=1;i<=5;i++)  
5     {  
6         for(j=1; j<=i; j++)  
7         {  
8             printf("*");  
9         }  
10        printf("\n");  
11    }  
12 }
```

WAP to print given pattern (nested loop)

1
12
123
1234
12345

No. of rows: 5

No. of values

Row-1: 1

Row-2: 12

Row-3: 123

Row-4: 1234

Row-5: 12345

Inner loop: Increment
Outer loop: Increment

Starting: 1

Program

```
1 void main()
2 {
3     int i,j;
4     for(i=1;i<=5;i++)
5     {
6         for(j=1; j<=i; j++)
7         {
8             printf("%d",j);
9         }
10        printf("\n");
11    }
12 }
```

WAP to print given pattern (nested loop)

5
54
543
5432
54321

No. of rows: 5

No. of values

Row-1: 5

Row-2: 54

Row-3: 543

Row-4: 5432

Row-5: 54321

Inner loop: Decrement

Outer loop:

Decrement/Increment

Starting: 5

Program

```
1 void main()
2 {
3     int i,j;
4     for(i=5;i>0;i--)
5     {
6         for(j=5; j>=i ; j--)
7         {
8             printf("%d",j);
9         }
10        printf("\n");
11    }
12 }
```

WAP to print given pattern (nested loop)

```
*
**
***
****
*****
```

No. of rows: 5

No. of values

Row-1: ----*

Row-2: ---**

Row-3: --***

Row-4: -****

Row-5: *****

Inner loop: Decrement

Outer loop: Decrement/Increment

Starting: -(space)

Ending: *

Program

```
1 void main()
2 {
3     int i,j,k;
4     for(i=1;i<=5;i++)
5     {
6         for(k=5;k>i;k--)
7         {
8             printf(" ");
9         }
10        for(j=1;j<=i;j++)
11        {
12            printf("*");
13        }
14        printf("\n");
15    }
16 }
```

First we need to print 4 spaces before printing *

```
*
**
***
****
*****
```

After printing spaces this inner loop prints *

Practice programs

- 1) Write a program to find sum of first N odd numbers. Ex. $1+3+5+7+\dots+N$
- 2) Write a program to find $1+1/2+1/3+1/4+\dots+1/n$.
- 3) Write a program to print all Armstrong numbers in a given range. For example $153 = 1^3 + 5^3 + 3^3$. So, 153 is Armstrong number.
- 4) Write a program to print given number in reverse order
- 5) Write a program to check whether a given string is palindrome or not.
- 6) Write a program to print Multiplication Table up to n.

1	2	3	4	5	6	7	.
2	4	6	8	10	12	14	.
3	6	9	12	15	18	21	.
4	8	12	16	20	24	28	.
5	10	15	20	25	30	35	.
.	.	.	.	.	.	.	.

- 7) Construct C programs to print the following patterns using loop statement.

```
1
22
333
4444
55555
```

```
*
# #
* * *
# # # #
* * * * *
```

```
1
0 1
1 0 1
0 1 0 1
```

```
1
2 2
3 3 3
4 4 4 4
```

```
1
A B
2 3 4
C D E F
```

```
* * * * *
*       *
*       *
* * * * *
```

```
* * * * *
* * * *
* * *
* *
*
```



Thank you

